

Course Outline: RabbitMQ Message Queues

Title: RabbitMQ: A Production Message Broker **Skill:** Skill 47: Using queues and tasks to delegate

Learnpack: + Implementación de colas de mensajes con RabbitMQ **File:** s47-w16d47-implementacion-de-colas-de-mensajes-con- **Prerequisite:** s47-w16d47-procesamiento-asincrono-con-colas-offloa

Target Audience: Beginner developers who understand async queue processing concepts and now need to implement with a production message broker **Total Lessons:** 9 **Format:** Mixed (11% hands-on = 1 lesson)

Course Description

RabbitMQ is the production face of everything you've learned about async queue processing. This course covers how RabbitMQ implements the producer-queue-consumer model with a specific architecture (producer → exchange → queue → consumer), how messages flow through exchanges and routing keys, how ACK and NACK guarantee reliable delivery, how to scale with multiple workers, and how to run the whole system with Docker. The hands-on challenge connects it all: publish a message with pika, consume it, and send an ACK.

Course Philosophy

This course emphasizes:

- The architecture first, the code second — RabbitMQ's components map directly to the async processing concepts already covered
 - ACK/NACK as the reliability mechanism that makes RabbitMQ production-grade
 - Docker as the zero-configuration path to running RabbitMQ for development
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - RabbitMQ Architecture	3
02 - Scaling and Deployment	3
03 - Assessment	1
04 - Outro	1
Total	9

00.0 RabbitMQ: A Production Message Broker 

Learning Objectives:

- Define RabbitMQ as a production-grade message broker implementing the AMQP protocol
- Connect RabbitMQ to the broader async queue processing concepts from the previous learnpack
- Preview the course structure

Content Outline:

- What RabbitMQ is: an open-source message broker that implements the Advanced Message Queuing Protocol (AMQP)
- RabbitMQ's position in the ecosystem: it's the implementation of the "message broker" concept covered in the previous learnpack (Skill 47 offset 6), providing durability, acknowledgment, and routing
- Why RabbitMQ specifically:
 - Widely used in production Python, Java, and Node.js applications
 - Rich routing capabilities (exchanges, routing keys, fanout)
 - Mature, stable, and well-documented
 - Easy to run locally via Docker
- The three components students already know from async processing: producer (creates messages), queue (holds messages), consumer (processes messages) — in RabbitMQ, an exchange sits between the producer and queue
- Course structure: Section 01 covers the architecture (how RabbitMQ works, message flow, ACK/NACK); Section 02 covers scaling and deployment (multiple workers, Docker, pika hands-on challenge)

Transitions: This lesson connects to prior knowledge. Section 01 introduces RabbitMQ's specific architecture.

Key Principles:

- RabbitMQ is not a different concept — it's a specific implementation of the message queue pattern with additional routing capabilities
- The AMQP protocol ensures interoperability: any language with an AMQP client can connect to RabbitMQ
- Learning RabbitMQ means learning the vocabulary (exchange, binding, routing key) that every AMQP-based system uses

Examples:

- Companies using RabbitMQ: Instagram (image processing), Mozilla (task distribution), GitHub Actions (CI/CD pipelines)
- Use case: a web app publishes "user.signup" events → RabbitMQ routes them → welcome email worker, analytics worker, and fraud detection worker each receive the event

01.0 RabbitMQ Architecture

Learning Objectives:

- Name the four core components of RabbitMQ: producer, exchange, queue, consumer
- Describe the role of each component using the postal system analogy
- Explain how the four components work together in a message delivery flow

Content Outline:

- RabbitMQ's postal system analogy:
 - **Producer** (sender): the application that sends messages — creates the letter
 - **Exchange** (post office): receives messages from producers and routes them to queues — the sorting office
 - **Queue** (mailbox): a buffer that stores messages until consumers are ready to process them — the mailbox
 - **Consumer** (worker): the application that receives and processes messages from a queue — the recipient
- The flow: Producer → Exchange → Queue → Consumer
- The binding: a queue is "bound" to an exchange with a routing key; the exchange uses this binding to decide which queue(s) receive which messages
- Exchange types (brief overview; this course focuses on the default direct exchange):
 - **Direct exchange**: routes messages to queues whose binding key exactly matches the message's routing key
 - **Fanout exchange**: routes messages to all bound queues, ignoring the routing key
 - **Topic exchange**: routes messages to queues based on routing key patterns
- The default exchange: in the simplest setup, the default exchange routes to a queue named by the routing key — `basic_publish(exchange='', routing_key='my_queue', ...)` sends directly to `my_queue`
- Section preview: 01.1 covers how messages actually flow through this architecture; 01.2 covers the ACK/NACK reliability mechanism

Transitions: You understand the components. 01.1 shows how a message travels through them.

Key Principles:

- The exchange is the key difference from the previous learnpack's simple queue — it adds routing intelligence between producer and queue
- The default exchange (empty string) is the simplest case: producer → queue directly, no custom routing
- Decoupling is even stronger in RabbitMQ: the producer doesn't name the queue — it names the exchange and routing key; the exchange decides which queue receives the message

Examples:

- Simple routing: `channel.basic_publish(exchange='', routing_key='emails', body='...')` → message goes to the 'emails' queue
 - Direct exchange: `exchange='notifications', routing_key='urgent'` → message goes to the 'urgent' queue (bound with key 'urgent')
 - Fanout: `exchange='events' (fanout), no routing key` → message goes to ALL queues bound to 'events'
-

01.1 How Messages Flow

Learning Objectives:

- Trace the complete path of a message from producer to consumer in RabbitMQ
- Explain the role of exchanges, routing keys, and bindings in message delivery
- Describe what happens when a message matches or doesn't match any binding

Content Outline:

- The complete message flow step by step:
 1. **Producer connects** to RabbitMQ via a connection (TCP) and channel (logical connection within TCP)
 2. **Producer publishes**: sends a message body + exchange name + routing key (+ optional properties like `delivery_mode`, `expiration`)
 3. **Exchange receives**: examines the routing key and applies its routing rules
 4. **Binding lookup**: exchange checks which queues are bound with a matching key
 5. **Message deposited**: exchange copies the message to each matching queue
 6. **Queue stores**: the message waits in the queue until a consumer is ready
 7. **Consumer dequeues**: consumer picks up the message from the queue
 8. **Consumer processes**: runs the application logic
 9. **ACK (or NACK)**: consumer signals success or failure to the broker
- What happens if no binding matches: the message is either dropped (default) or returned to the producer (if `mandatory=True`)
- Message properties:
 - `delivery_mode=2` (persistent): message survives a RabbitMQ restart — stored to disk
 - `delivery_mode=1` (transient): message lives in memory only
 - `expiration`: time in milliseconds before an unprocessed message is discarded
- The channel concept: channels are lightweight virtual connections multiplexed over a single TCP connection — most applications use one channel per thread/consumer
- Section preview: 01.2 covers ACK and NACK — the mechanism that makes step 9 matter for reliability

Transitions: You can trace a message's full journey. 01.2 explains what happens at the final step: acknowledgment.

Key Principles:

- The message flow is deterministic: given the exchange type, routing key, and bindings, you can always predict which queues receive a message
- Persistent messages (`delivery_mode=2`) survive broker restarts — always use this for production jobs
- The channel is the unit of work: one channel per consumer avoids thread-safety issues

Examples:

```
# Producer side
channel.queue_declare(queue='task_queue', durable=True)
```

```
channel.basic_publish(
    exchange='', # default exchange
    routing_key='task_queue', # routing to this queue directly
    body=json.dumps({'task': 'send_email', 'to': 'user@example.com'}),
    properties=pika.BasicProperties(
        delivery_mode=pika.DeliveryMode.Persistent # survive restarts
    )
)
```

01.2 ACK and NACK

Learning Objectives:

- Define ACK (acknowledgment) as the consumer's signal that a message was processed successfully
- Define NACK (negative acknowledgment) as the consumer's signal that a message was NOT processed successfully
- Explain what RabbitMQ does with unacknowledged and negatively-acknowledged messages

Content Outline:

- **The problem ACK solves:**
 - Without acknowledgment: RabbitMQ removes the message from the queue the moment it is delivered to a consumer
 - If the consumer crashes after receiving but before finishing, the message is lost
 - With acknowledgment: RabbitMQ keeps the message "in-flight" (marked as unacknowledged) until it receives explicit confirmation
- **ACK (positive acknowledgment):**
 - Sent by the consumer after successfully processing a message
 - Tells RabbitMQ: "I processed this message, you can delete it from the queue"
 - Code: `ch.basic_ack(delivery_tag=method.delivery_tag)`
 - `delivery_tag` is a unique identifier for the message delivery within the channel
- **NACK (negative acknowledgment):**
 - Sent by the consumer when processing fails
 - Tells RabbitMQ: "I could not process this message; put it back in the queue (or discard it)"
 - Code: `ch.basic_nack(delivery_tag=method.delivery_tag, requeue=True)`
 - `requeue=True`: message goes back to the front of the queue for another consumer to try
 - `requeue=False`: message is discarded or sent to the dead letter exchange
- **Auto-ack vs manual ack:**
 - Auto-ack (`no_ack=True` or `auto_ack=True`): RabbitMQ removes the message immediately on delivery — no reliability guarantee
 - Manual ack: consumer explicitly sends ACK when done — production standard
- **Prefetch count (`basic_qos`):**
 - Limits how many messages RabbitMQ delivers to a consumer at once before it sends ACK
 - `channel.basic_qos(prefetch_count=1)`: deliver one message at a time; wait for ACK before delivering the next

- Prevents a fast consumer from taking all messages and a slow consumer from getting none

Transitions: ACK/NACK is the reliability heart of RabbitMQ. Section 02 covers how to scale with multiple workers and how to set up and run RabbitMQ.

Key Principles:

- Always use manual acknowledgment in production — auto-ack means lost messages on consumer crash
- `basic_qos(prefetch_count=1)` ensures fair dispatch: workers only receive new messages after they've processed and acknowledged the previous one
- NACK with `requeue=False` + a dead letter exchange is the production pattern for handling poison pill messages

Examples:

```
def callback(ch, method, properties, body):
    try:
        process(json.loads(body))
        ch.basic_ack(delivery_tag=method.delivery_tag)    # success → delete
    except Exception as e:
        log_error(e)
        ch.basic_nack(delivery_tag=method.delivery_tag, requeue=False) #
# fail → DLQ
```

02.0 Scaling and Deployment

Learning Objectives:

- Understand how RabbitMQ distributes work across multiple worker processes
- Know what Docker command starts a RabbitMQ instance with the management plugin
- Preview Section 02: multiple workers, Docker setup, and the coding challenge

Content Outline:

- **Scaling with RabbitMQ:** the same principle as the worker pattern from Skill 47 offset 6, applied to RabbitMQ
 - Multiple consumers connected to the same queue
 - RabbitMQ dispatches messages in round-robin order (with `basic_qos(prefetch_count=1)`, in fair dispatch order)
 - Each message is delivered to exactly one consumer
- **Fair dispatch vs round-robin:**
 - Round-robin (without prefetch limit): RabbitMQ pre-fetches several messages for each consumer; a slow consumer blocks while messages pile up; a fast consumer gets no new messages

- Fair dispatch (with `basic_qos(prefetch_count=1)`): RabbitMQ only delivers a new message to a consumer that has sent an ACK for the previous one; naturally routes work to the fastest available worker
- **Docker setup:**
 - Command: `docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-management`
 - Port 5672: AMQP protocol port — for pika connections
 - Port 15672: Management UI — a browser interface at `http://localhost:15672` (username/password: guest/guest)
 - The management plugin provides real-time visibility into queues, messages, and consumers — invaluable for debugging
- Section preview: 02.1 covers multiple workers in detail; 02.2 is the coding challenge

Transitions: This overview sets up the practical section. 02.1 explains worker scaling; 02.2 puts it all together in code.

Key Principles:

- `basic_qos(prefetch_count=1)` is the single most important setting for fair load distribution in RabbitMQ
- The management UI (port 15672) is the fastest way to verify that your producer is publishing and your consumers are consuming
- Docker makes running RabbitMQ locally trivially easy — no installation required

Examples:

```
# Start RabbitMQ with management UI
docker run -d --name rabbitmq \
  -p 5672:5672 \
  -p 15672:15672 \
  rabbitmq:3-management

# Check it's running
docker ps # should show rabbitmq container

# Open http://localhost:15672 in browser (guest/guest)
```

02.1 Multiple Workers

Learning Objectives:

- Explain how RabbitMQ distributes messages across multiple workers with fair dispatch
- Describe the competing consumers pattern
- Identify the role of `basic_qos` in preventing uneven load distribution

Content Outline:

- **Competing consumers pattern:**

- Multiple consumers connected to the same queue
- They "compete" to dequeue messages — each message goes to exactly one consumer
- This is the standard pattern for horizontal scaling: add more worker processes to increase throughput
- **How RabbitMQ dispatches messages to multiple workers:**
 - Without `basic_qos`: round-robin at the prefetch level — messages are pre-allocated to workers before they start processing; a slow worker receives future messages it can't process yet while a fast worker sits idle
 - With `basic_qos(prefetch_count=1)`: RabbitMQ only sends the next message to a worker after it has ACKed the previous one — automatically routes work to whichever worker is available
- **Practical implications:**
 - Starting a second worker doubles throughput (assuming the bottleneck is processing, not RabbitMQ)
 - Workers can be started and stopped without affecting the queue — messages wait if no workers are available
 - Worker crashes don't lose messages — unacknowledged messages are redelivered to another worker
- **Multiple workers for no bottlenecks (from the curriculum):** this is exactly the competing consumers pattern — each worker is independent, so no single worker is a bottleneck
- Real-world example: a photo processing pipeline with 10 workers can process 10 photos in parallel; adding 10 more workers doubles the throughput to 20 parallel operations

Transitions: You understand how workers scale. 02.2 puts everything together with the coding challenge.

Key Principles:

- The competing consumers pattern is the canonical way to scale RabbitMQ-based systems
- Workers are fully interchangeable — any worker can process any message from the queue
- `basic_qos(prefetch_count=1)` is required for fair distribution; without it, some workers may be overloaded

Examples:

```
# Both of these workers connect to the same queue
# Worker 1 in process 1:
channel.basic_consume(queue='task_queue', on_message_callback=callback)
channel.start_consuming()

# Worker 2 in process 2 (identical code):
channel.basic_consume(queue='task_queue', on_message_callback=callback)
channel.start_consuming()
# → RabbitMQ distributes messages between them based on availability
```


Challenge Prompt: Start RabbitMQ with Docker, then write a Python script using pika that declares a durable queue named `task_queue`, publishes a persistent message `{"task": "process_data", "data": "hello world"}` to it, and then consumes the message with a callback that prints the received message, sends an ACK, and stops consuming. The script should print structured log entries at each step.

Solution Code:

```
import pika
import json

# Connect to RabbitMQ
connection = pika.BlockingConnection(
    pika.ConnectionParameters('localhost')
)
channel = connection.channel()

# Declare a durable queue
channel.queue_declare(queue='task_queue', durable=True)

# Publish a persistent message
message = {"task": "process_data", "data": "hello world"}
channel.basic_publish(
    exchange='',
    routing_key='task_queue',
    body=json.dumps(message),
    properties=pika.BasicProperties(
        delivery_mode=pika.DeliveryMode.Persistent
    )
)
print(f"Published: {json.dumps(message)}")

# Consumer callback
def callback(ch, method, properties, body):
    received = json.loads(body)
    print(f"Received: {received}")
    ch.basic_ack(delivery_tag=method.delivery_tag)
    print("ACK sent")
    ch.stop_consuming()

# Consume one message with fair dispatch
channel.basic_qos(prefetch_count=1)
channel.basic_consume(queue='task_queue', on_message_callback=callback)
print("Waiting for messages...")
channel.start_consuming()

connection.close()
print("Done")
```

Challenge Code:

```

import pika
import json

# Connect to RabbitMQ running on localhost (BlockingConnection)
# your code here

# Declare a durable queue named 'task_queue'
# your code here

# Publish a persistent message: {"task": "process_data", "data": "hello world"}
# Print f"Published: {json.dumps(message)}"
# your code here

# Define a callback function that:
# - Deserializes the body from JSON
# - Prints f"Received: {received}"
# - Sends ACK with ch.basic_ack(delivery_tag=method.delivery_tag)
# - Prints "ACK sent"
# - Stops consuming with ch.stop_consuming()
# your code here

# Set prefetch_count=1, start consuming from 'task_queue'
# Print "Waiting for messages..." before start_consuming()
# your code here

# Close the connection and print "Done"
# your code here

```

03.0 Assessment 🧐

Learning Objectives:

- Apply RabbitMQ architecture knowledge to new scenarios
- Identify the correct RabbitMQ component for each role in a message flow
- Diagnose reliability issues in described RabbitMQ configurations

Question Format: Scenario-based (multiple-choice)

Questions:

1. In RabbitMQ's postal system analogy, which component acts as the "post office sorting room" that routes messages to the correct queues?
 - A) Producer
 - B) Queue
 - C) Exchange ✓
 - D) Consumer

2. You publish a message with `delivery_mode=1`. What happens to this message if RabbitMQ restarts?

- A) The message is redelivered to the consumer after restart
- B) The message is lost because it is stored only in memory ✓
- C) The message is moved to the dead letter queue
- D) The message is automatically re-sent by the producer

3. A consumer receives a message and crashes before calling `basic_ack()`. What does RabbitMQ do?

- A) The message is permanently deleted from the queue
- B) The message is logged as failed
- C) The message remains in the queue and is redelivered to another available consumer ✓
- D) The producer is notified to re-send the message

4. Which pika setting ensures that RabbitMQ distributes work fairly among multiple workers, sending each only one unacknowledged message at a time?

- A) `channel.basic_consume(auto_ack=True)`
- B) `channel.basic_qos(prefetch_count=1)` ✓
- C) `channel.queue_declare(exclusive=True)`
- D) `pika.ConnectionParameters(heartbeat=0)`

5. You have 10 workers connected to the same `task_queue`. A message is published. How many workers receive it?

- A) All 10 workers receive a copy
- B) Exactly 1 worker receives it ✓
- C) The first 3 workers receive it (round-robin to multiple)
- D) None — RabbitMQ requires explicit worker assignment

6. You call `ch.basic_nack(delivery_tag=method.delivery_tag, requeue=False)`. What happens to the message?

- A) It is re-added to the front of the queue
- B) It is sent back to the producer
- C) It is discarded or sent to the dead letter exchange ✓
- D) It is held for manual review by an administrator

7. Which Docker port must be exposed for your pika Python code to connect to RabbitMQ?

- A) 15672 (management UI)
- B) 5672 (AMQP protocol) ✓
- C) 3306 (MySQL)
- D) 6379 (Redis)

8. You declare a queue with `durable=True`. What does this guarantee?

- A) Messages in the queue are automatically acknowledged

- B) The queue survives a RabbitMQ restart ✓
- C) Multiple consumers can share the queue
- D) Messages are delivered in strict priority order

Evaluation Criteria:

- Correctly identifies each component's role in the architecture
- Understands the reliability implications of ACK and delivery_mode
- Applies fair dispatch knowledge to multiple worker scenarios
- Recognizes Docker ports and durable queue semantics

Score Interpretation:

- 8/8: Ready to implement RabbitMQ-based systems in production
 - 6–7: Review any missed lessons before moving on
 - 4–5: Re-read Sections 01 and 02
 - Below 4: Restart from Section 01
-

04.0 RabbitMQ in Your Stack

Content Outline:

- Course recap: what students accomplished
 - Learned RabbitMQ's four components: producer → exchange → queue → consumer
 - Traced the complete message flow from publish to ACK
 - Understood ACK and NACK as the reliability mechanism that prevents message loss on consumer crash
 - Ran RabbitMQ with Docker and connected with pika
 - Used `basic_qos(prefetch_count=1)` for fair work distribution across multiple workers
 - Published a persistent message and consumed it with manual acknowledgment
- Connection to bigger picture: RabbitMQ is one implementation of the AMQP protocol. Every concept you learned here (exchanges, routing keys, ACK, prefetch) transfers to other AMQP-based systems. The vocabulary is also shared with cloud broker services (AWS SQS has equivalent concepts for visibility timeout and dead letter queues).
- Next steps: The remaining learnpacks move into machine learning and predictions. The background processing toolkit is now complete — from understanding what background processing is (Skill 45) to queue fundamentals (Skill 46) to async patterns (Skill 47 offset 6) to this hands-on RabbitMQ implementation. You can now build, deploy, and scale async queue-based systems.
- Resources for further exploration:
 - RabbitMQ official tutorials (rabbitmq.com/tutorials) — six progressive tutorials covering all exchange types
 - pika documentation — Python client library
 - RabbitMQ management UI (localhost:15672) — learn what metrics to monitor in production
 - "RabbitMQ in Action" (book) — the definitive guide

- Encouragement: RabbitMQ is used by companies processing millions of messages per day. What you've built and understood in this course is the same architecture, at the same conceptual level, as those production systems. The difference is scale — and scale is just more of the same: more workers, more queues, more exchanges. You now have the foundation to grow into that.

Note: This is conclusion only — no theory, exercises, or assessment.